

CortexOS 当前 RTOS 技术总览与模块完成度

1. 文档目的

本文档用于系统性说明当前 CortexOS 的：

- 项目定位
- 整体结构
- 运行形态
- 核心功能
- 各模块的设计原理
- 各模块的当前实现方式
- 各模块的完成度与剩余缺口

本文档面向的读者是：

- 新加入项目的开发者
- 需要快速理解当前 RTOS 状态的非项目负责人
- 后续接手做功能扩展、工程验证或科研收束的人员

如果只需要快速上手使用，请看 [README.md](#) 和 [docs/release/USER_GUIDE.md](#)。如果需要更偏向代码入口和开发路线的说明，请看 [docs/release/DEVELOPER_GUIDE.md](#)。本文档更强调“当前系统到底已经做到了什么、原理是什么、完成到什么程度”。

2. 项目定位

CortexOS 是一个面向**低端 Cortex-M 开发板**的 `no_std` RTOS 工程原型，当前重点不是做通用操作系统，而是做一个：

- 可在无 **MPU/MMU** 的低端单片机上运行
- 具备静态内存、固定容量、可预测执行路径
- 可支持多板扩展
- 可承载控制类应用样板
- 后续可作为 Rust RTOS 安全性研究载体

当前代表板卡：

- `board-f411-nucleo`
- `board-f103c8-bluepill`
- `f103rct6-generic` (**F103** 的别名板配置)

当前系统主用途：

- 运行默认 UART 控制 APP
 - 执行 bench 基准测试（当前仅承诺 **F411**）
 - 作为后续雕刻机 / AGV / 机械臂控制样板的底座
-

3. 当前系统的总体结构

3.1 结构分层

当前代码已经形成明确的分层：

层次	主要路径	作用
架构层	src/arch/cortex_m/	Cortex-M 启动、PendSV、CPU 指令、异常控制
内核层	src/kernel.rs、src/task/、src/timer/、src/sync/、src/ipc/	调度、时间、同步、通信、诊断
平台门面层	src/platform/	向应用层暴露统一的串口、控制、诊断、看门狗能力
设备抽象层	src/device/	GPIO/PWM/UART/ADC/Timer 等板无关封装
BSP 层	src/bsp/	各板 HAL/PAC 绑定、资源映射、启动资源接管
应用层	src/app.rs、src/app_protocol.rs	默认 APP 的协议、任务、业务逻辑
基准测试层	src/bench.rs	bench 固件与性能测试逻辑

3.2 基本依赖方向

当前工程的正确依赖方向是：

- app -> platform/device
- platform -> bsp/device
- kernel -> platform facade
- bsp -> HAL/PAC

这意味着：

- 应用层不应直接接触 HAL/PAC
- 内核层不应直接绑定某块板子的专属驱动实现
- 板级差异主要被限制在 bsp 及少量配置层中

这是当前多板扩展和后续安全封装的基础。

4. 当前支持的运行形态

4.1 默认 APP 固件

默认固件的主要职责是：

- 接收串口指令

- 解析 ASCII 行协议
- 执行 `PING / ECHO / LED / PWM / STAT`
- 回传执行结果与健康状态

其任务链路是：

`UART RX IRQ -> ring buffer -> uart_rx_task -> 命令队列 -> app_cmd_task -> TX 队列 -> uart_tx_task`

这一路径已经是当前系统最成熟的工程样板。

4.2 bench 固件

`bench` 固件用于测量当前 RTOS 的：

- 上下文切换
- 信号量唤醒
- 队列唤醒
- IRQ 到任务延迟
- 软定时器回调延迟
- timeout wheel
- scheduler scale / O(1) 行为

当前 `bench` 只承诺 `board-f411-nucleo`，并不作为 `F103` 的正式能力。

4.3 uart-probe 固件

工程中还存在 `uart-probe` feature，用于更小范围地验证某些 UART 路径。这不是默认主线运行形态，而是辅助调试形态。

5. 当前 RTOS 的核心功能概览

5.1 已具备的核心能力

当前已经完成并可用的核心功能包括：

- 任务创建、启动、删除、退出
- 基于优先级位图 + ready queue 的调度
- 时间片轮转
- 阻塞 / 睡眠 / 超时唤醒
- timeout wheel 超时管理
- `SysTick` 节拍推进
- 软定时器
- 互斥、信号量、事件
- 固定容量消息队列和 ring buffer
- 任务心跳、栈水位、trace counters、reset reason
- 看门狗协调
- 双板 BSP 与多板脚本构建链路

5.2 当前系统边界

当前系统有明确边界：

- 不使用堆作为默认系统基础设施
- 不提供用户态 / 内核态隔离
- 不提供文件系统、网络栈
- 不以高端 SoC 或 Linux 平台为当前主线
- 不把更多板卡数量作为当前价值主线

因此，它当前更准确的描述是：

面向低端板控制场景的 Rust RTOS 工程底座。

6. 模块逐项说明

下面按模块说明“原理 / 当前实现 / 完成度”。

6.1 `src/arch/cortex_m/` —— 架构与上下文切换层

作用

该层负责 Cortex-M 启动、异常入口和上下文切换，是整个 RTOS 的最低层。

原理

Cortex-M RTOS 的核心切换机制通常基于：

- `SysTick`：提供系统节拍
- `PendSV`：作为最低优先级上下文切换异常
- PSP/MSP：区分线程上下文和异常上下文

本工程延续这一标准设计：

- `SysTick` 触发调度决策
- 真正的寄存器保存/恢复由 `PendSV` 汇编完成
- Rust 调度器只负责决定“切换到谁”

当前实现

涉及文件：

- `src/arch/cortex_m/boot.S`
- `src/arch/cortex_m/pendsv.S`
- `src/arch/cortex_m/cpu.rs`
- `src/arch/cortex_m/interrupts.rs`

实现状态：

- 启动路径可用
- `PendSV` 上下文切换可用
- 与 Rust 调度器的联动已经稳定支撑默认 APP 和 bench

- 面向 Cortex-M3/M4 的主路径已可工作

完成度

- **完成度：高**
 - 现阶段已足以作为主线底座使用
 - 后续主要缺口不在功能，而在：
 - safe/unsafe 边界梳理
 - 架构层 trusted code base 的文档化
-

6.2 src/kernel.rs —— 内核门面层

作用

kernel.rs 是应用层和 RTOS 核心之间的统一入口，用于屏蔽底层调度器、定时器和诊断实现细节。

原理

该层的设计原则是：

- 应用只调用 kernel API
- 应用不直接操作调度器内部状态
- kernel 向上提供“够用且稳定”的任务、定时器、诊断和健康接口

这使应用开发不需要理解所有底层实现细节。

当前实现

当前已提供：

- 任务生命周期 API：创建、启动、yield、sleep、block、unblock、delete、exit
- 优先级与查询 API
- 诊断与 trace API
- 心跳与系统健康 API
- 看门狗门面
- 软定时器门面

完成度

- **完成度：高（面向当前工程范围）**
- 对默认 APP 和 bench 已经足够
- 后续仍可能增加更高层控制类 API，但不影响其作为内核门面的稳定地位

主要后续工作：

- 继续避免 kernel 反向依赖具体板实现
 - 为未来安全研究建立更清晰的 TCB 边界
-

6.3 src/task/ —— 任务、TCB 与调度器

作用

该模块负责：

- 任务控制块（TCB）管理
- 就绪队列管理
- 阻塞 / 睡眠 / 唤醒状态转换
- 优先级调度
- trace/diagnostics

原理

当前调度策略基于：

- `priority bitmap`
- 每优先级一个 `ready queue`
- `timeout wheel` 管理超时唤醒
- `idle` 任务不进入普通 `ready queue`

这样做的目的，是尽量让“选择下一个运行任务”的路径维持在接近 $O(1)$ 的复杂度。

当前实现

关键文件：

- `src/task/scheduler.rs`
- `src/task/tcb.rs`
- `src/task/context.rs`
- `src/task/diagnostics.rs`

当前已具备：

- 固定任务表
- $O(1)$ ready path
- 时间片调度
- 阻塞/睡眠/超时唤醒
- `timeout wheel`
- `trace counters`
- 任务诊断与栈水位
- `bench` 所需的调度观测点

完成度

- **完成度：高**
- 从工程可用性看，当前调度器已可支撑默认 APP、bench、双板运行
- 从研究与安全性角度看，仍有后续工作：
 - `unsafe` 使用点收口
 - 调度关键全局状态的封装强化
 - 更正式的 TCB 说明

6.4 `src/timer/` —— 时间与定时器系统

作用

该模块负责：

- 系统时间推进
- `SysTick` 节拍
- 软定时器
- `bench` 所需的硬件定时器桥接

原理

时间系统由两个部分构成：

- `SysTick`：提供主系统 tick
- `soft_timer`：在 tick 基础上实现任务级定时回调

在 `F411 bench` 场景下，还引入了专用硬件定时器路径用于更细的基准测量。

当前实现

- `systick.rs`：系统 tick 读写、tick 与 ms 换算、延时辅助
- `soft_timer.rs`：软定时器注册、到期触发、周期重装
- `hw_timer.rs`：当前主要服务于 `F411 bench` 场景

完成度

- `SysTick`：高
- `soft_timer`：高
- `hw_timer`：中（当前偏 `bench` 支撑，不是双板通用运行时基础设施）

总体评价：

- 时间系统对当前默认 APP 和 `bench` 已经够用
- 如果后续控制场景需要更强的硬实时外设定能力，还需要在阶段 2 及之后继续扩展

6.5 `src/sync/` —— 同步原语

作用

该模块提供任务之间和中断/任务之间协作所需的同步机制。

当前组件

- `IrqMutex`
- `BlockingMutex`
- `Semaphore`
- `Event`

原理

设计原则是：

- 中断只做最小工作
- 唤醒与调度尽量交给任务上下文完成
- 数据结构固定容量，不引入动态分配

当前实现

- `IrqMutex`：用于轻量临界区保护
- `BlockingMutex`：支持任务阻塞等待，bench 中也用于优先级继承路径测试
- `Semaphore`：支持任务等待/唤醒与超时
- `Event`：支持广播式信号通知

完成度

- **完成度：高**
- 已经具备当前 APP 和 bench 所需能力
- 剩余工作主要不在功能缺失，而在：
 - 后续控制场景中继续验证接口是否足够稳定
 - 安全性研究时梳理其内部 unsafe 与调度耦合点

6.6 `src/ipc/` —— 进程间/任务间通信

作用

提供固定容量的轻量通信机制，用于串口数据链路和任务之间消息传递。

当前组件

- `ringbuf_core.rs`：无锁式固定容量字节环形缓冲区核心
- `ringbuf.rs`：IRQ-safe 封装
- `mqueue.rs`：固定容量消息队列

原理

当前 IPC 的设计目标不是通用大而全，而是：

- 满足默认 APP 的串口字节流缓存
- 满足命令槽索引、bench 消息传递这类小消息场景
- 所有结构静态定长、行为可预测

当前实现

- `ringbuf` 用于 UART RX/TX 链路
- `mqueue` 用于命令索引、bench 协作消息等
- host tests 中已覆盖协议解析与 ring buffer 边界用例

完成度

- **完成度：高（面向当前场景）**
 - 对默认 APP 非常成熟
 - 对更复杂的大对象/多生产者多消费者场景还未扩展，这属于后续业务阶段的工作
-

6.7 `src/platform/` —— 平台门面层

作用

该层向应用暴露平台级统一能力，同时把板差异和设备细节藏在下面。

当前子模块

- `platform::uart`
- `platform::controls`
- `platform::diagnostics`
- `platform::watchdog`

原理

`platform` 的目标是：

- 让应用层只关心“能做什么”
- 不关心“是哪块板、哪个 UART、哪个 GPIO”

这是多板扩展和后续 safe-first API 的关键位置。

当前实现

- `uart`：区分 `BootConsole` 与 `AppUart` 角色
- `controls`：统一 LED / PWM 能力
- `diagnostics`：统一 I/O 健康快照
- `watchdog`：统一看门狗启动与喂狗入口

完成度

- **完成度：高**
 - 当前是工程中最关键的边界层之一
 - 已经足以支撑双板默认 APP
 - 后续阶段 1 的重点之一就是继续冻结这层基础接口
-

6.8 `src/device/` —— 板无关设备封装

作用

对底层设备能力做更基础、更贴近硬件语义的封装，使其能被 `platform` 和应用复用。

当前组件

- `uart.rs`
- `gpio.rs`
- `pwm.rs`
- `adc.rs` (当前 F411)
- `timer.rs` (当前偏 F411)

原理

`device` 不是 BSP，本层不负责板资源选择，而负责：

- 把同类能力整理成统一的 Rust 接口
- 降低 HAL 类型直接向上传播的范围

当前实现

- `uart`：直接委托到 `platform::uart`，用于 APP/日志访问
- `gpio`：输出/输入包装
- `pwm`：占空比控制包装
- `adc`：F411 的 ADC 简单包装
- `timer`：F411 的系统/硬件定时器包装

完成度

- UART / GPIO / PWM：高
- ADC：中（已存在，但不是当前主线功能）
- Timer device：中（主要服务已有实现和 bench，不是全面通用设备层）

总体上，`device` 已经达到“够用且能继续演进”的状态，但还没有发展成非常完整的嵌入式驱动框架。

6.9 `src/bsp/` —— 板支持包层

作用

BSP 负责：

- 接管具体芯片/板子的初始化
- 建立 UART / LED / PWM / watchdog 等板级资源映射
- 输出统一的 `BoardContext`
- 向 `platform` / `device` 提供板级实现

当前实现

当前包含：

- `f411_nucleo.rs`
- `f103c8_bluepill.rs`
- `mod.rs` 负责按 feature 选出 `current`

F411 路径

特点：

- 使用 `stm32f4xx-hal`
- 已完成 APP / bench / soak 主线验证
- 系统时钟配置为 `84MHz`
- UART、LED、PWM、IWDG、bench 相关资源已经齐全

F103 路径

特点：

- 使用 `stm32f1`
- 以保守配置为主，当前主频固定 `8MHz`
- 已完成 APP / smoke / soak 主线验证
- bench 不支持
- 部分硬件接管使用了更直接的寄存器级实现，反映出其仍偏“可用优先”

完成度

- F411 BSP：高
- F103 BSP：中高
- 多板框架本身：高

当前限制：

- 正式承诺板卡仍只有少数代表板
- F103 的实现相对更保守，也更接近“资源受限板”的现实状态

6.10 `src/app.rs` 与 `src/app_protocol.rs` —— 默认应用层

作用

这是当前对外最完整的运行样板，用来证明 RTOS 不只是能切任务，还能承载实际控制型 APP。

原理

默认 APP 采用“中断最小化、任务处理主逻辑”的设计：

- 中断只负责把数据放入缓冲区并触发事件
- `uart_rx_task` 负责组帧
- `app_cmd_task` 负责命令解析和业务执行
- `uart_tx_task` 负责统一发送
- `health_task` 负责健康上报和看门狗协调

协议采用简单的 ASCII 行协议，方便联调、串口观察和脚本测试。

当前实现

当前支持命令：

- PING
- ECHO <text>
- LED ON|OFF|TOGGLE
- PWM <0-100>
- STAT

已具备：

- 固定容量命令池
- 超长行丢弃
- F103 UART 错误时整行丢弃的快速抗干扰修复
- 状态与健康回传

完成度

- **完成度：高（作为默认样板）**
- 已具备很好的工程演示价值
- 但它仍然只是“基础控制 APP 模板”，不是下一阶段雕刻机桥接业务的最终形态

后续演进方向：

- 从通用示例命令集升级为控制桥接样板
- 引入更明确的控制命令、转发链路和状态回传模型

6.11 src/bench.rs —— 基准测试层

作用

bench 固件用于回答“当前 RTOS 的关键路径性能如何”。

原理

bench 通过：

- DWT cycle counter
- TIM2 辅助路径
- 多个专门的任务和同步对象

来对关键延迟进行重复采样，并输出统计结果。

当前覆盖指标

包括但不限于：

- context switch
- semaphore latency
- queue wake / end-to-end latency
- irq-to-task latency

- mutex / priority inheritance
- soft timer callback latency
- timeout 验证
- scheduler scaling
- scheduler O(1) 检查

完成度

- **完成度：高（仅 F411）**
 - 对当前研究和工程调优价值很高
 - 对 F103 暂不提供正式支持，这是明确的当前边界
-

6.12 src/mem/ —— 内存辅助模块

作用

提供一些静态内存相关能力与布局辅助。

当前实现

- `layout.rs`：提供内存布局信息与栈起始位置
- `static_pool.rs`：固定块大小的静态池分配器

当前定位

这一层不是当前系统的核心运行路径，而是一个“可用于后续扩展的辅助基础设施”。

完成度

- **完成度：中**
 - 已实现，但当前默认 APP 和主调度路径并不重度依赖它
 - 后续如果要做更复杂的静态对象管理，它会更有价值
-

6.13 src/driver/ —— 简单驱动包装层

作用

提供更贴近控制场景的简单驱动包装。

当前组件

- `motor.rs`
- `encoder.rs`
- `sensor.rs`

原理

这些模块不是板级驱动，而是建立在 `device` 层之上的“小而轻”的控制对象包装，例如：

- 电机：PWM + 方向控制
- 编码器：计数封装
- 数字传感器：输入 pin 封装

当前实现

实现都比较轻量，偏“基础构件”，尚未形成完整的控制系统中间层。

完成度

- **完成度：中低**
- 代码已经存在且可复用
- 但还没有进入默认 APP 主链路，也还不是当前双板验收的核心组成部分
- 更适合在后续雕刻机 / AGV / 机械臂样板阶段被真正接入

6.14 src/log.rs 与辅助模块

作用

提供启动期与运行期日志输出支撑。

当前实现

当前日志主要通过 UART 输出，能支撑：

- boot banner
- bench 输出
- 默认 APP 状态信息
- 健康信息与故障信息

完成度

- **完成度：中高**
- 作为嵌入式串口日志已够用
- 若未来需要更系统化的日志级别/结构化事件，还可继续扩展

7. 当前模块完成度总表

模块	当前状态	完成度判断	说明
arch/cortex_m	已支撑双板运行	高	后续重点在安全边界，不在功能缺失
kernel	API 可用、职责清晰	高	已能支撑 APP/bench
task	调度、超时、诊断成熟	高	后续重点是 unsafe/TCB 收口
timer	tick/soft timer 稳定	高/中	hw_timer 偏 F411 bench
sync	原语完整	高	已满足当前工程需要

模块	当前状态	完成度判断	说明
ipc	ring buffer / queue 稳定	高	默认 APP 强依赖，已成熟
platform	边界已成形	高	是后续阶段 1 收口重点之一
device	核心设备能力可用	中高	UART/GPIO/PWM 高，ADC/Timer 较局部
bsp	双板已跑通	高 / 中高	F411 更成熟，F103 更保守
app	默认控制 APP 可落板	高	但仍是基础样板，不是最终业务形态
bench	F411 完整	高（单板）	明确不承诺 F103
mem	存在辅助设施	中	当前不是主线核心
driver	控制构件已存在	中低	待后续业务样板真正接入
log	串口日志可用	中高	满足当前工程需求

8. 当前系统的整体完成度判断

如果从不同角度看，当前 RTOS 的完成度可以分开评价。

8.1 作为“能运行的 RTOS 原型”

- **完成度：高**

理由：

- 已具备任务、调度、同步、IPC、时间、诊断、双板 BSP
- 已能长稳运行默认 APP
- 已具备 bench 和多板回归链路

8.2 作为“可承载工程控制样板的底座”

- **完成度：中高**

理由：

- 底座能力已基本具备
- 默认 APP 已证明串口控制链路可工作
- 但还未正式落到雕刻机桥接等真实业务样板

8.3 作为“面向科研的 Rust 安全 RTOS 研究样本”

- **完成度：中**

理由：

- 工程底座已经足够支撑研究
- 但真正的 `safe/unsafe` 收口、TCB 边界和安全性论证还未展开

因此，当前项目最准确的阶段判断是：

工程底座已基本成型，正在从“可运行系统”过渡到“可承载真实控制样板、可进一步做安全研究”的阶段。

9. 当前剩余缺口

为了让读者形成完整认识，这里明确列出当前仍未完成或未定版的内容。

9.1 工程层面

- 阶段 1 的结构/接口/脚本进一步收口
- **F103 UART** 长时抗干扰持续观察
- 文档与实现的持续同步维护

9.2 应用层面

- 雕刻机控制桥接样板
- 面向 AGV / 机械臂的控制样板
- 更贴近真实工程的通信协议

9.3 科研层面

- **unsafe inventory**
 - TCB 边界文档
 - **kernel/task** deeper safe/unsafe 重构
 - Rust 安全收益的正式研究论证
-

10. 结论

当前 **Cortex0S** 已经不是一个“只有调度器和几个 demo”的最小 RTOS 练习工程，而是一个已经具备：

- 双板运行能力
- 默认应用样板
- bench 测试链路
- 多板构建/测试脚本
- 平台门面与 BSP 分层
- 基本健康监控与长稳验证

的嵌入式 RTOS 工程底座。

它当前最合适的定位不是“功能已经全部完成”，而是：

RTOS 底座已经完成到足以进入真实控制样板开发，并能继续向 Rust 安全性研究收束的阶段。